

# **LAPORAN PRAKTIKUM**

## **TEKNOLOGI IOT II**

*Komunikasi Asinkron dan WebSockets, Topologi Jaringan Terdesentralisasi  
(ESP Now dan Mesh)*



Disusun Oleh:

Faiz Hisyam Rasyid (25782068)

Dosen Pengajar:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET**

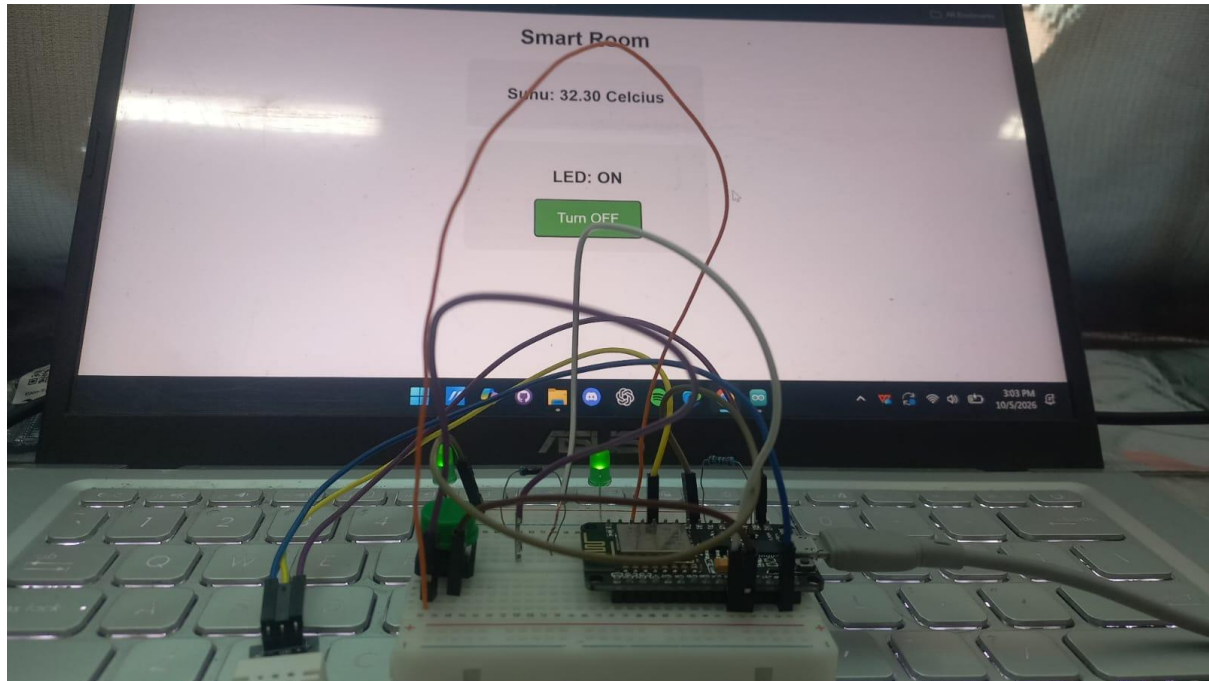
**JURUSAN TEKNOLOGI INFORMASI**

**POLITEKNIK NEGERI LAMPUNG**

**2026/2027**

## Modul 4: Komunikasi Asinkron dan WebSockets

### 4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik



```
Blink | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
Blink.ino
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "Aku suka kamu";
7 const char* password = "Kamusukasamayanglain";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT22);
15
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;

Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM12')
.....
IP Address: 192.168.1.32
```

**Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

HTTP Refresh Bekerja dengan cara klien memutus dan membuka kembali koneksi HTTP secara berulang-ulang. Setiap kali memuat data (misal setiap 3 detik), browser harus mengunduh ulang seluruh struktur dokumen HTML mentah, meskipun yang berubah hanya 1 angka suhu.

WebSockets Koneksi TCP dibuat hanya melakukan *handshake* awal sekali. Setelah pipa komunikasi terbuka, pertukaran data berlangsung secara *full-duplex*. Saat ada pembaruan data, NodeMCU hanya mengirimkan paket teks kecil berformat JSON (hanya puluhan byte, misal {"suhu": "28"}). Tidak ada overhead header HTTP berulang maupun kerangka HTML yang dikirim ulang.

**Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Memanggil fungsi `delay(3000)` di dalam `loop()` akan menyebabkan pembekuan kerja CPU ESP8266 selama tiga detik penuh karena mikrokontroler ini hanya memiliki satu inti pemroses (*single-core*). Hal ini akan membuat, mulai dari tombol fisik yang menjadi tidak responsif karena pembacaan status pin dan ikut tertahan, hingga perintah sakelar dari antarmuka web yang mengalami jeda waktu parah. Selain itu, jeda pemblokiran yang terlalu lama dapat menggagalkan pertukaran paket penjaga koneksi (*heartbeat/ping-pong*), sehingga *browser* menganggap sambungan WebSocket telah terputus (*timeout*) dan terus-menerus mencoba melakukan rekoneksi ulang.

**Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Pada blok kode JavaScript khusus yang merombak antarmuka tombol dan teks status perangkat keras, elemen HTML bertag *span* yang menjadi target injeksi teks adalah elemen dengan atribut ID `ledStatus`, di mana teks di dalamnya diubah menjadi "ON" atau "OFF" sesuai kondisi logika LED. Elemen lain yang disentuh pada blok tersebut adalah `toggleBtn`, namun elemen itu bertipe tombol (*button*), bukan *span*. Jika ditinjau

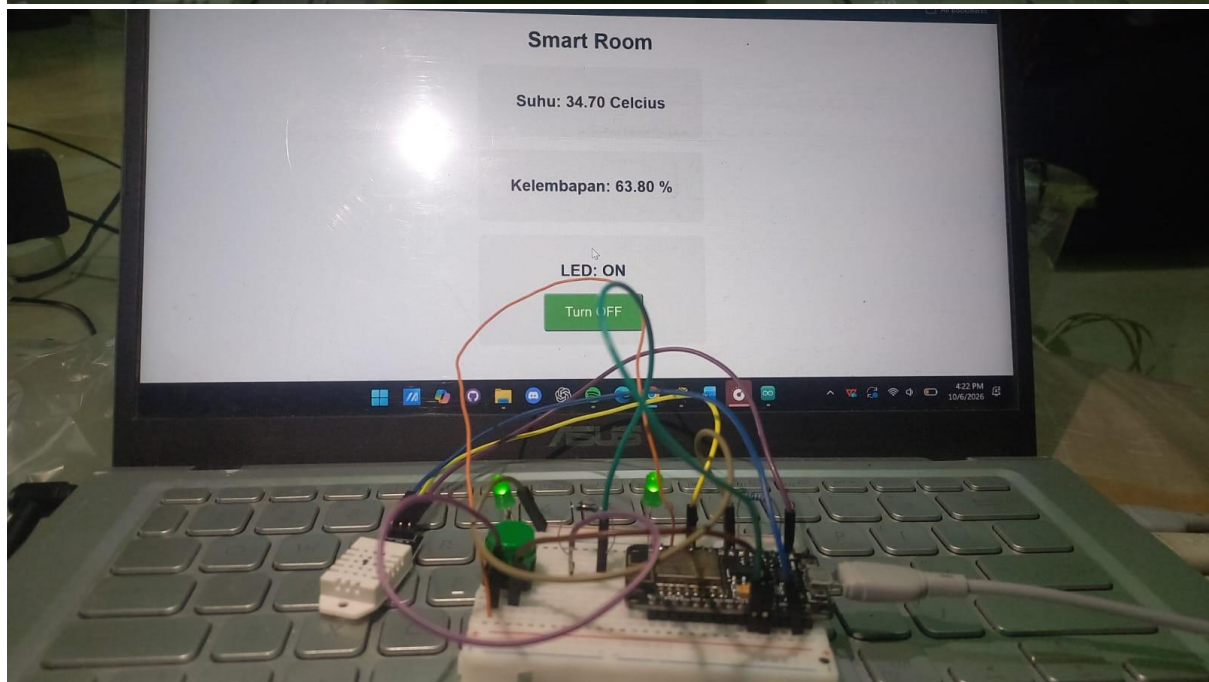
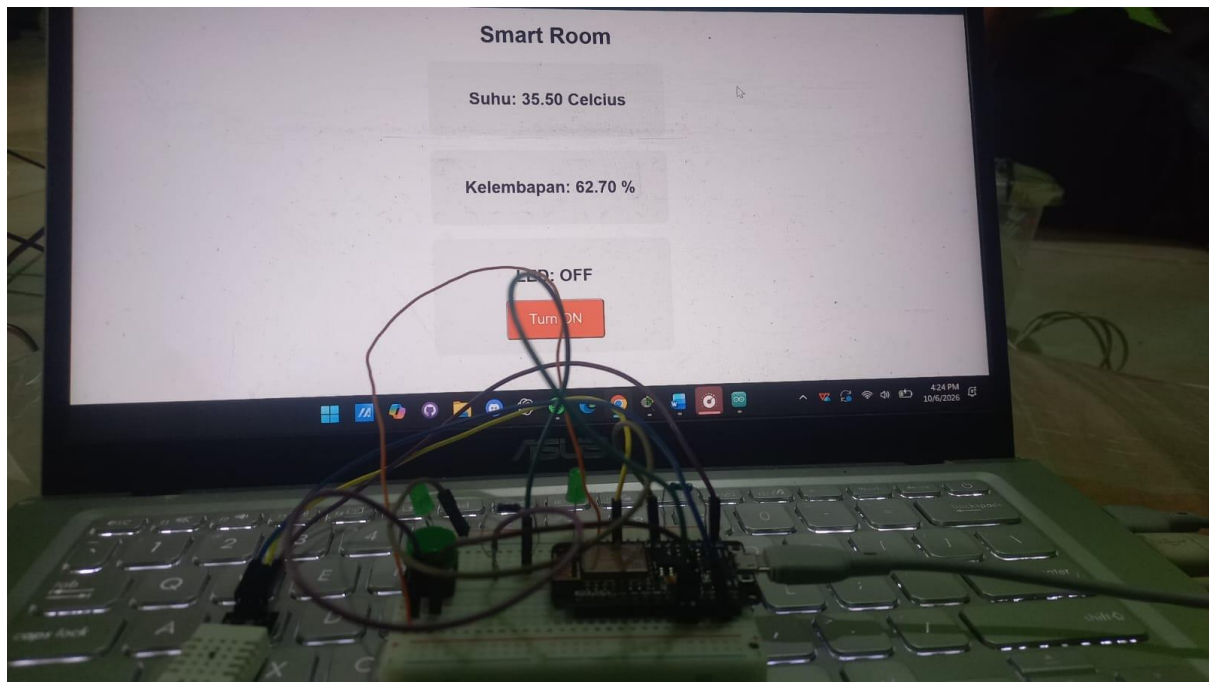
pada fungsi `onMessage` secara keseluruhan, terdapat satu elemen bertag *span* lainnya yang ditargetkan untuk penyuntikan angka suhu, yaitu elemen dengan ID `tempValue`

**Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

- i. **Sisi Back-End (C++):** Buat variabel penampung (misal: `String currentHum = "--";`). Baca nilai kelembapan menggunakan fungsi `dht.readHumidity()` di dalam siklus timer *non-blocking* `millis()`. Perbarui fungsi `notifyClients()` agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: `{"led": "...", "suhu": "...", "hum": "..."}.`
- ii. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (`<div class="card">`) baru pada kerangka HTML sebagai penampil kelembapan (misal: `<h2>Kelembapan: <span id="humValue">--</span> %</h2>`). Kemudian, modifikasi fungsi JavaScript `onMessage(event)` agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen `humValue`. Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

```
Blink | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
Blink.ino
205 // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
206 if ((millis() - lastTime) > 3000) {
207   float t = dht.readTemperature();
208   float h = dht.readHumidity(); // Membaca nilai kelembapan
209
210   bool adaUpdate = false;
211   if(!isnan(t)) {
212     currentTemp = String(t);
213     adaUpdate = true;
214   }
215   if(!isnan(h)) {
216     currentHum = String(h); // Menyimpan nilai kelembapan
217     adaUpdate = true;
218   }
219
220   if(adaUpdate) {
221     notifyClients(); // Push pembaruan suhu & kelembapan otomatis
222   }
223   lastTime = millis();
224 }
225 }

Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM12')
.
IP Address: 192.168.1.32
Client WebSocket #1 terhubung
```



### Bagian pemanbahan code

```
16 // Variabel Pelacak Status (State & Cache)
17 bool ledState = false;
18 String currentTemp = "--";
19 String currentHum = "--";    // Penambahan variabel kelembapan
20 int buttonState;
21 int lastButtonState = LOW;
22 unsigned long lastDebounceTime = 0;
23 const unsigned long debounceDelay = 50;
24 unsigned long lastTime = 0;
```

```

46 <h1>Smart Room</h1>
47 <div class="card">
48   <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
49 </div>
50 <!-- Penambahan kartu tampilan kelembapan -->
51 <div class="card">
52   <h2>Kelembapan: <span id="humValue">--</span> %</h2>
53 </div>
54 <div class="card">
55   <h2>LED: <span id="ledStatus">OFF</span></h2>
56   <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
57 </div>

```

```

87 // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
88 if(dataObj.suhu !== undefined) {
89   document.getElementById('tempValue').innerHTML = dataObj.suhu;
90 }
91
92 // Penambahan injeksi angka Kelembapan ke dalam id "humValue" HTML
93 if(dataObj.hum !== undefined) {
94   document.getElementById('humValue').innerHTML = dataObj.hum;
95 }

```

```

117 // ----- BACK-END & WEBSOCKET LOGIC -----
118
119 // Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
120 void notifyClients() {
121   String jsonString = "{\"led\": \"" + String(ledState ? 1 : 0) + "\", ";
122   jsonString += "\"suhu\": \"" + currentTemp + "\", ";
123   jsonString += "\"hum\": \"" + currentHum + "\"}"; // Penambahan atribut hum pada JSON
124   ws.textAll(jsonString);
125 }

```

```

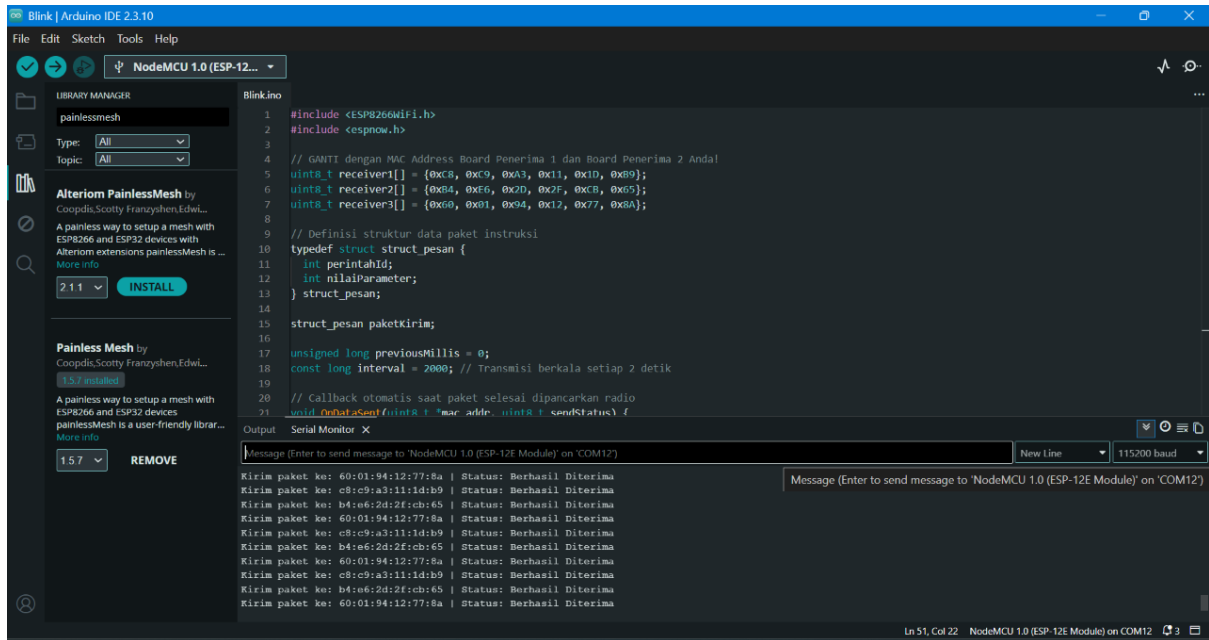
205 // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
206 if ((millis() - lastTime) > 3000) {
207   float t = dht.readTemperature();
208   float h = dht.readHumidity(); // Membaca nilai kelembapan
209
210   bool adaUpdate = false;
211   if(!isnan(t)) {
212     currentTemp = String(t);
213     adaUpdate = true;
214   }
215   if(!isnan(h)) {
216     currentHum = String(h); // Menyimpan nilai kelembapan
217     adaUpdate = true;
218   }
219
220   if(adaUpdate) {
221     notifyClients(); // Push pembaruan suhu & kelembapan otomatis
222   }
223   lastTime = millis();
224 }

```

## Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

### 5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

#### Controller



```
1 #include <ESP8266WiFi.h>
2 #include <ESPNow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver1[] = {0x18, 0xc9, 0xa3, 0x11, 0xd, 0xb9};
6 uint8_t receiver2[] = {0x18, 0xc9, 0xa3, 0x11, 0xd, 0xb9};
7 uint8_t receiver3[] = {0x18, 0xc9, 0xa3, 0x11, 0xd, 0xb9};
8
9 // Definisi struktur data paket instruksi
10 typedef struct struct_pesanan {
11     int perintahId;
12     int nilaiParameter;
13 } struct_pesanan;
14
15 struct_pesanan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long interval = 2000; // Transmisi berkala setiap 2 detik
19
20 // Callback otomatis saat paket selesai dipancarkan radio
21 void OnDataSent(uint8_t *mac_addr, uint8_t *data, uint8_t len) {
22     Serial.println("Paket dikirim ke: " + String(mac_addr) + " | Status: Berhasil Diterima");
23 }
```

Serial Monitor X

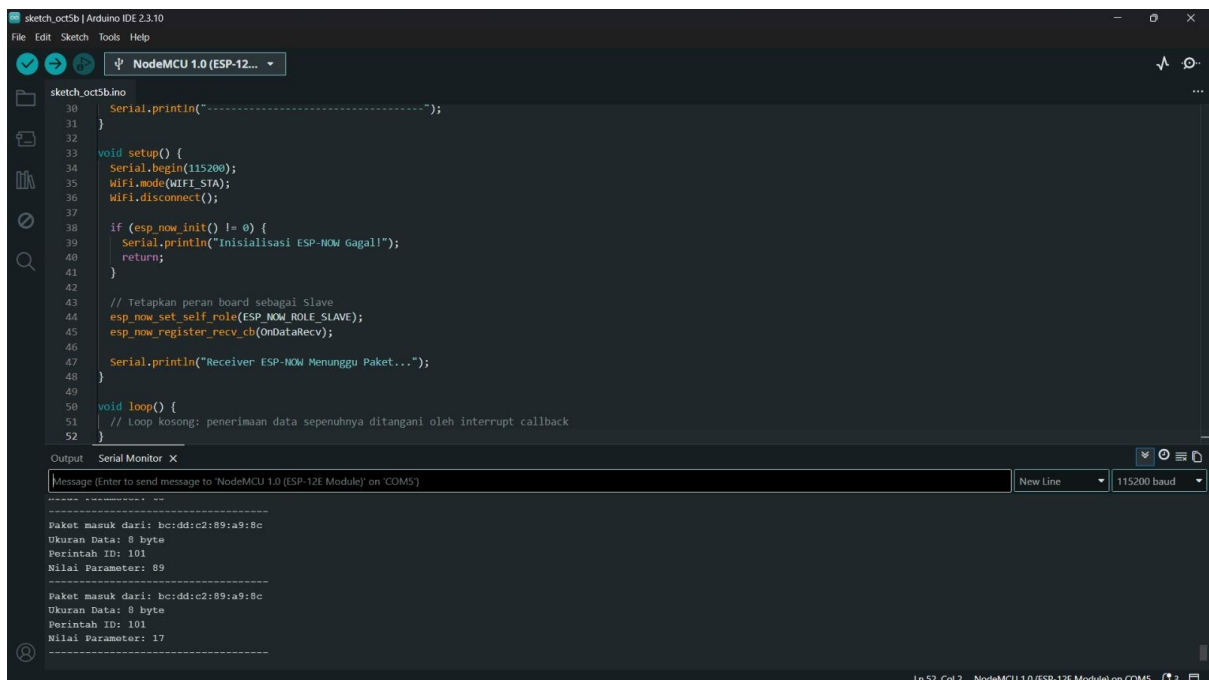
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM12')

New Line 115200 baud

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM12')

Kirim paket ke: 60:01:94:12:77:8a | Status: Berhasil Diterima  
Kirim paket ke: c8:c9:a3:11:d:b9 | Status: Berhasil Diterima  
Kirim paket ke: b4:e6:2d:2f:cb:65 | Status: Berhasil Diterima  
Kirim paket ke: 60:01:94:12:77:8a | Status: Berhasil Diterima  
Kirim paket ke: c8:c9:a3:11:d:b9 | Status: Berhasil Diterima  
Kirim paket ke: b4:e6:2d:2f:cb:65 | Status: Berhasil Diterima  
Kirim paket ke: 60:01:94:12:77:8a | Status: Berhasil Diterima  
Kirim paket ke: c8:c9:a3:11:d:b9 | Status: Berhasil Diterima  
Kirim paket ke: b4:e6:2d:2f:cb:65 | Status: Berhasil Diterima  
Kirim paket ke: 60:01:94:12:77:8a | Status: Berhasil Diterima

#### Penerima 1



```
30 Serial.println("-----");
31 }
32
33 void setup() {
34     Serial.begin(115200);
35     WiFi.mode(WIFI_STA);
36     WiFi.disconnect();
37
38     if (esp_now_init() != 0) {
39         Serial.println("Inisialisasi ESP-NOW Gagal!");
40         return;
41     }
42
43     // Tetapkan peran board sebagai Slave
44     esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
45     esp_now_register_recv_cb(OnDataRecv);
46
47     Serial.println("Receiver: ESP-NOW Menunggu Paket...");
48 }
49
50 void loop() {
51     // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
52 }
```

Serial Monitor X

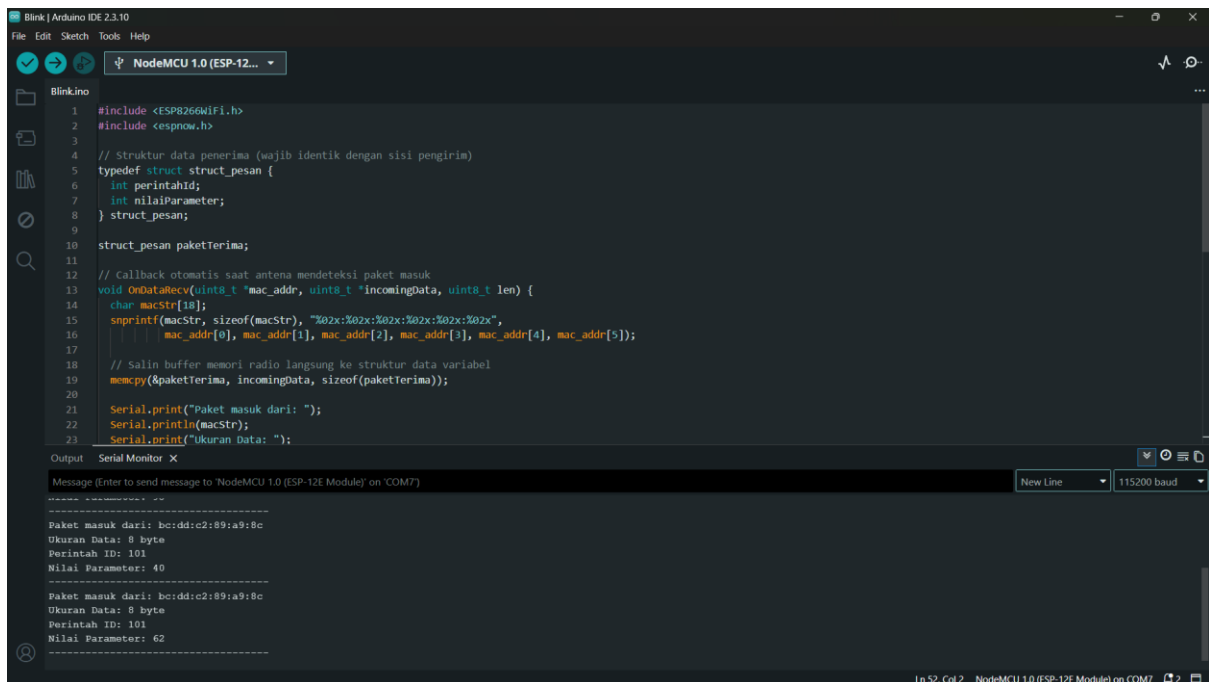
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

New Line 115200 baud

Paket masuk dari: bc:dd:c2:89:a9:8c  
Ukuran Data: 8 byte  
Perintah ID: 101  
Nilai Parameter: 89

Paket masuk dari: bc:dd:c2:89:a9:8c  
Ukuran Data: 8 byte  
Perintah ID: 101  
Nilai Parameter: 17

## Penerima 2



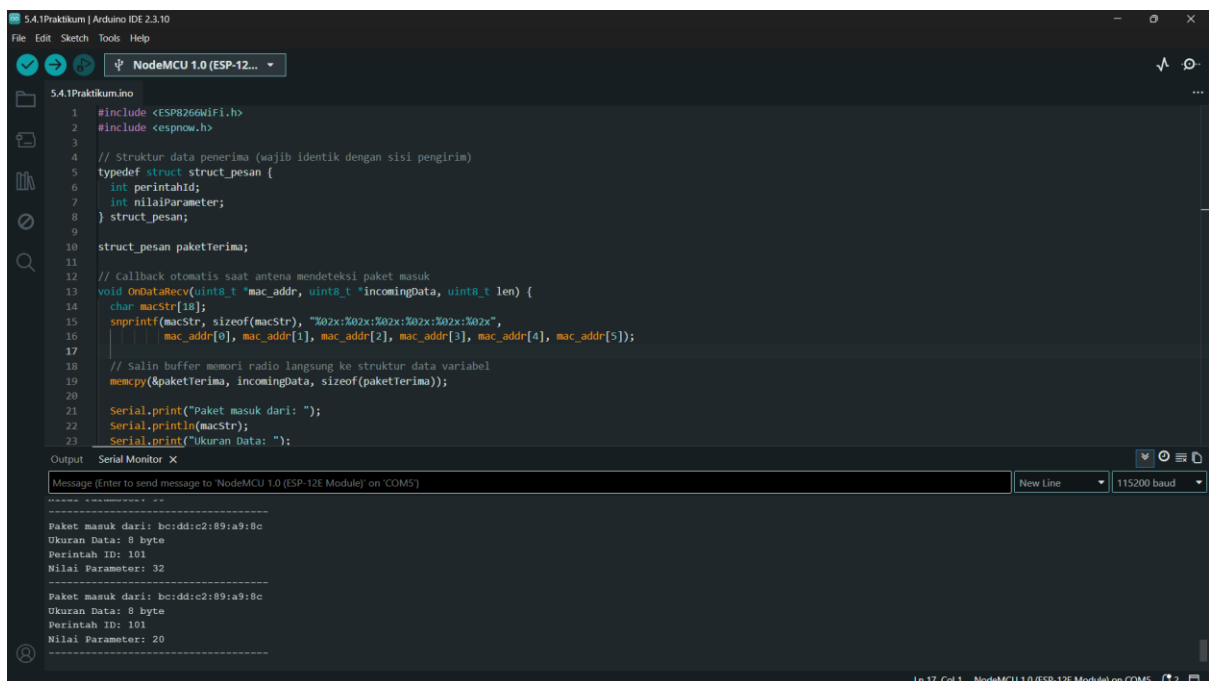
The screenshot shows the Arduino IDE interface with the file 'Blinkino.ino' open. The code defines a structure for received data and implements a callback function to process incoming packets. The serial monitor shows two received packets with their respective MAC addresses, sizes, command IDs, and parameters.

```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahid;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
19     memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21     Serial.print("Paket masuk dari: ");
22     Serial.println(macStr);
23     Serial.print("Ukuran Data: ");
```

Serial Monitor Output:

```
-----
Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 40
-----
Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 62
-----
```

## Penerima 3



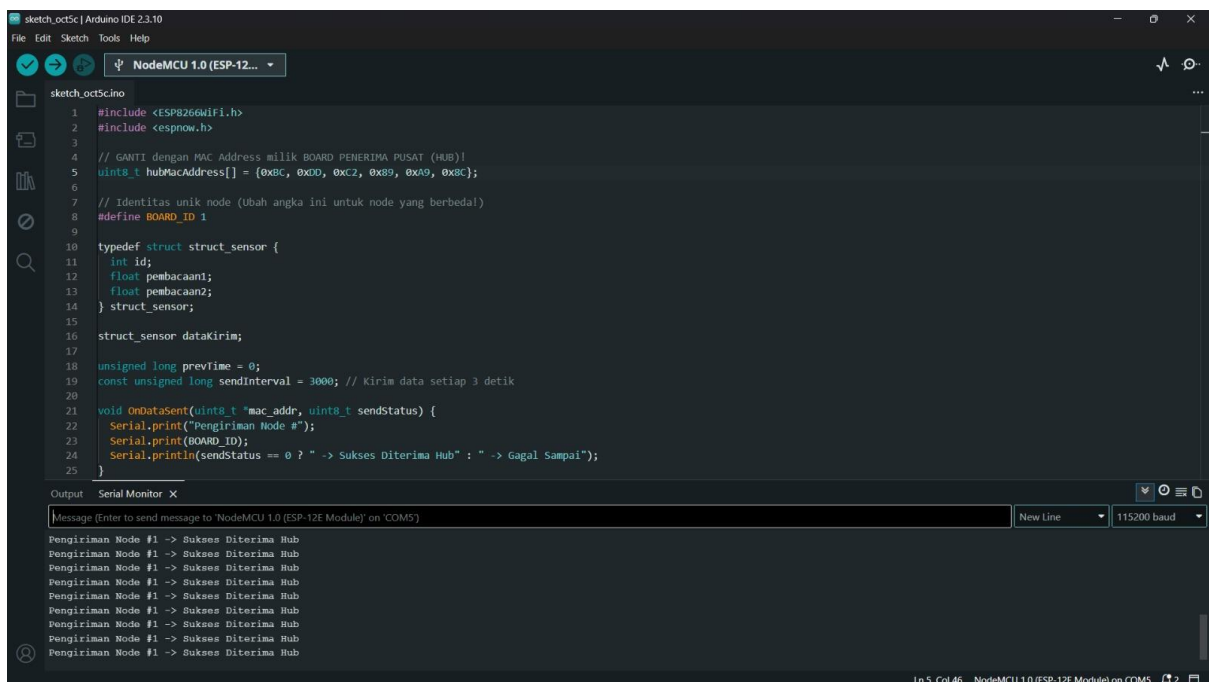
The screenshot shows the Arduino IDE interface with the file '5.4.1Praktikum.ino' open. The code is identical to the one in the first screenshot. The serial monitor shows two received packets with their respective MAC addresses, sizes, command IDs, and parameters.

```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahid;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
19     memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21     Serial.print("Paket masuk dari: ");
22     Serial.println(macStr);
23     Serial.print("Ukuran Data: ");
```

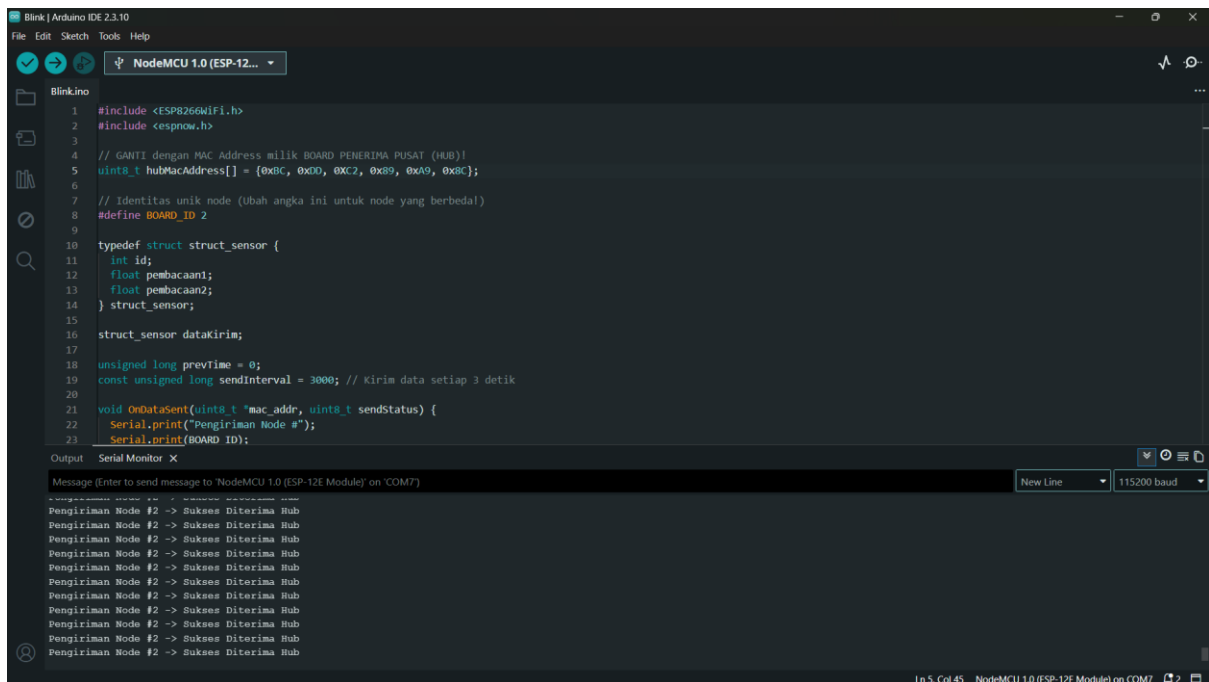
Serial Monitor Output:

```
-----
Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 32
-----
Paket masuk dari: bc:dd:c2:89:a9:8c
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 20
-----
```

## Controller



## Penerima 2



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)
5 uint8_t hubMacAddress[] = {0xBC, 0x00, 0xC2, 0xA9, 0xA9, 0x8C};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda)
8 #define BOARD_ID 2
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24 }
```

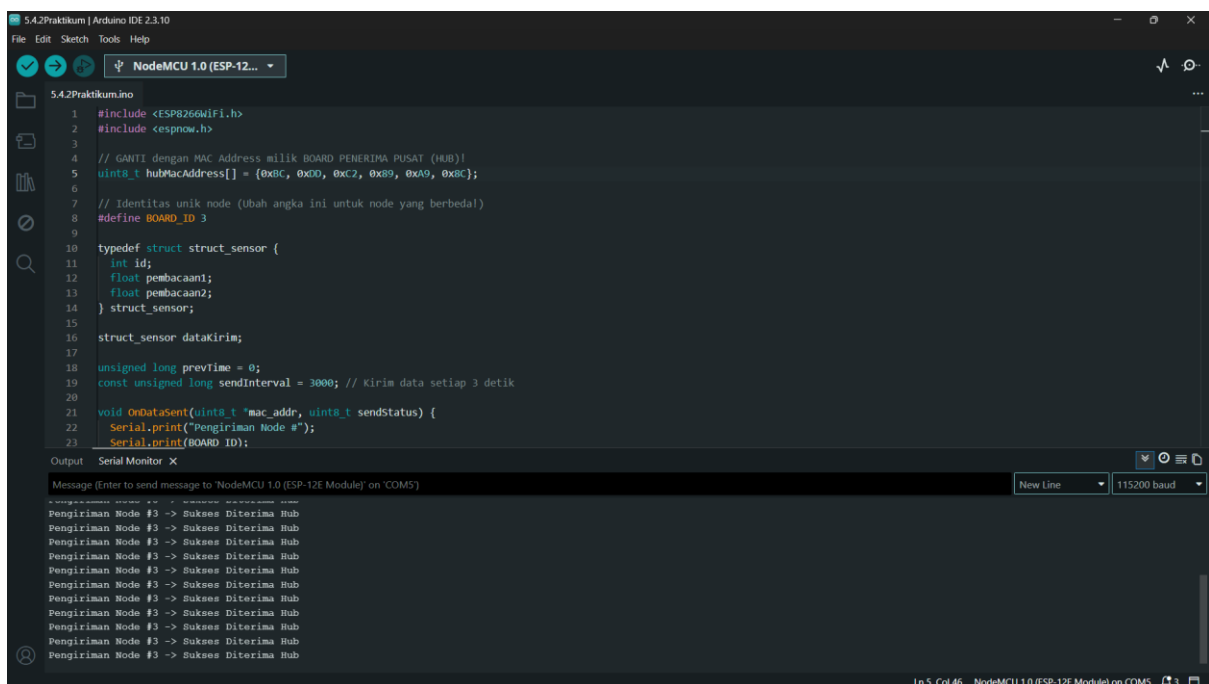
Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub  
Pengiriman Node #2 -> Sukses Diterima Hub

Ln 5, Col 45 NodeMCU 1.0 (ESP-12E Module) on COM7

## Penerima 3



```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)
5 uint8_t hubMacAddress[] = {0xBC, 0x00, 0xC2, 0xA9, 0xA9, 0x8C};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda)
8 #define BOARD_ID 3
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void onDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");
23     Serial.print(BOARD_ID);
24 }
```

Output Serial Monitor X

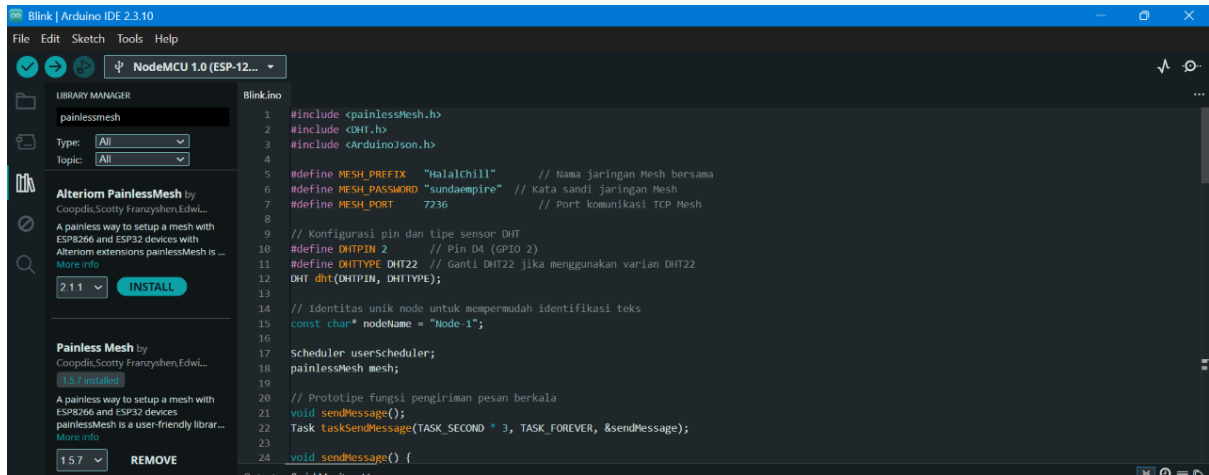
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub  
Pengiriman Node #3 -> Sukses Diterima Hub

Ln 5, Col 46 NodeMCU 1.0 (ESP-12E Module) on COM5

## 5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

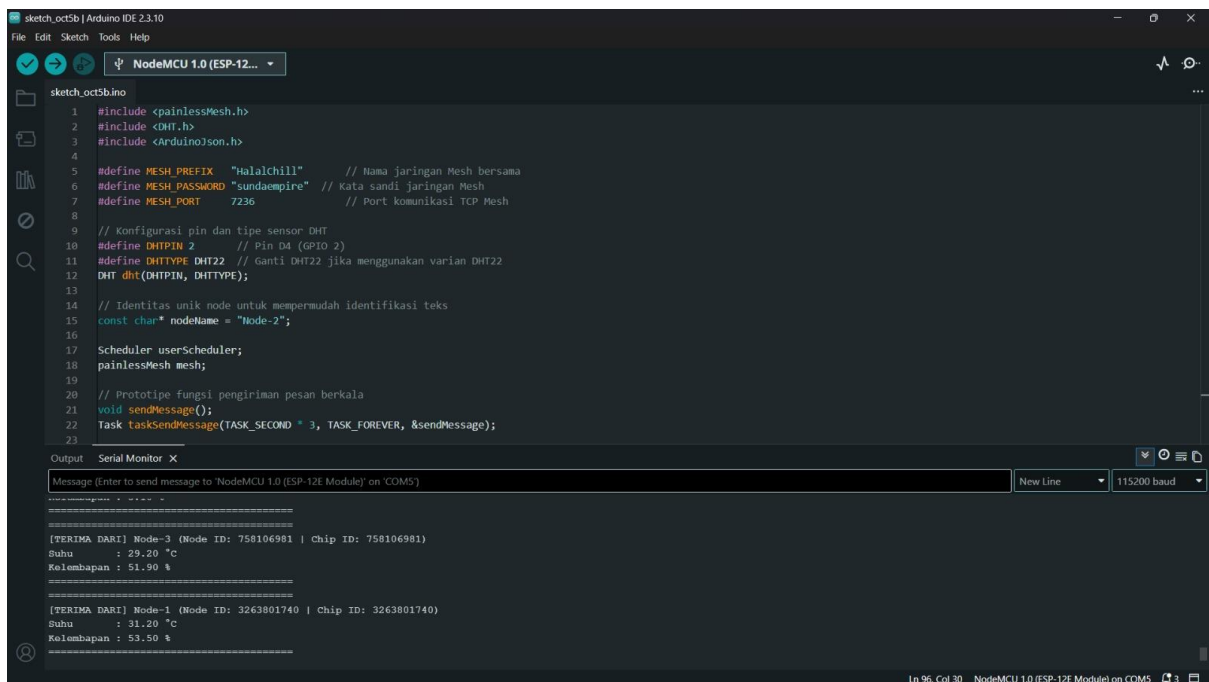
### Controller



The screenshot shows the Arduino IDE interface with the 'Controller' sketch loaded. The sketch is for a NodeMCU 1.0 (ESP-12E) and uses the 'painlessMesh' library. The code includes headers for 'painlessMesh.h', 'DHT.h', and 'ArduinoJson.h'. It defines mesh parameters: MESH\_PREFIX 'Halalchill', MESH\_PASSWORD 'sundaempire', and MESH\_PORT 7236. It configures the DHT sensor on pin 2 (DHT22). The node is identified as 'Node-1'. The code sets up a scheduler and a periodic message task. The Serial Monitor is open at the bottom.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Halalchill" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "sundaempire" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7236 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 Scheduler usersScheduler;
18 painlessMesh mesh;
19
20 // Prototype fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
```

### Penerima 1



The screenshot shows the Arduino IDE interface with the 'Receiver 1' sketch loaded. The sketch is for a NodeMCU 1.0 (ESP-12E) and uses the 'painlessMesh' library. It includes the same headers and mesh parameters as the Controller sketch. It configures the DHT sensor on pin 2 (DHT22). The node is identified as 'Node-2'. The code sets up a scheduler and a periodic message task. The Serial Monitor is open at the bottom, showing the output of the sketch.

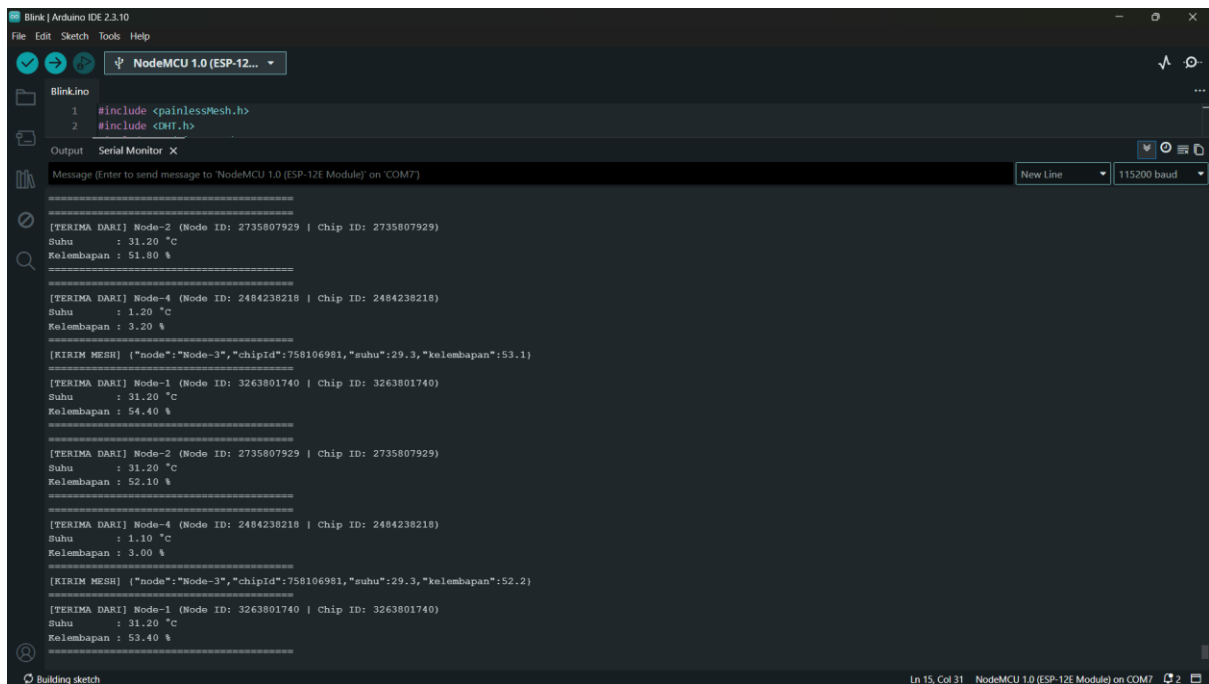
```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Halalchill" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "sundaempire" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7236 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2";
16
17 Scheduler usersScheduler;
18 painlessMesh mesh;
19
20 // Prototype fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
```

Serial Monitor Output:

```
[TERIMA DARI] Node-3 (Node ID: 758106981 | Chip ID: 758106981)
Suhu : 29.20 °C
Kelembapan : 51.90 %

[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)
Suhu : 31.20 °C
Kelembapan : 53.50 %
```

## Penerima 2



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')

=====

[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)  
Suhu : 31.20 °C  
Kelembapan : 51.80 %

=====

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)  
Suhu : 1.20 °C  
Kelembapan : 3.20 %

=====

[KIRIM MESH] ["node":"Node-3","chipid":758106981,"suhu":29.3,"kelembapan":53.1)

=====

[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)  
Suhu : 31.20 °C  
Kelembapan : 54.40 %

=====

[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)  
Suhu : 31.20 °C  
Kelembapan : 52.10 %

=====

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)  
Suhu : 1.10 °C  
Kelembapan : 3.00 %

=====

[KIRIM MESH] ["node":"Node-3","chipid":758106981,"suhu":29.3,"kelembapan":52.2)

=====

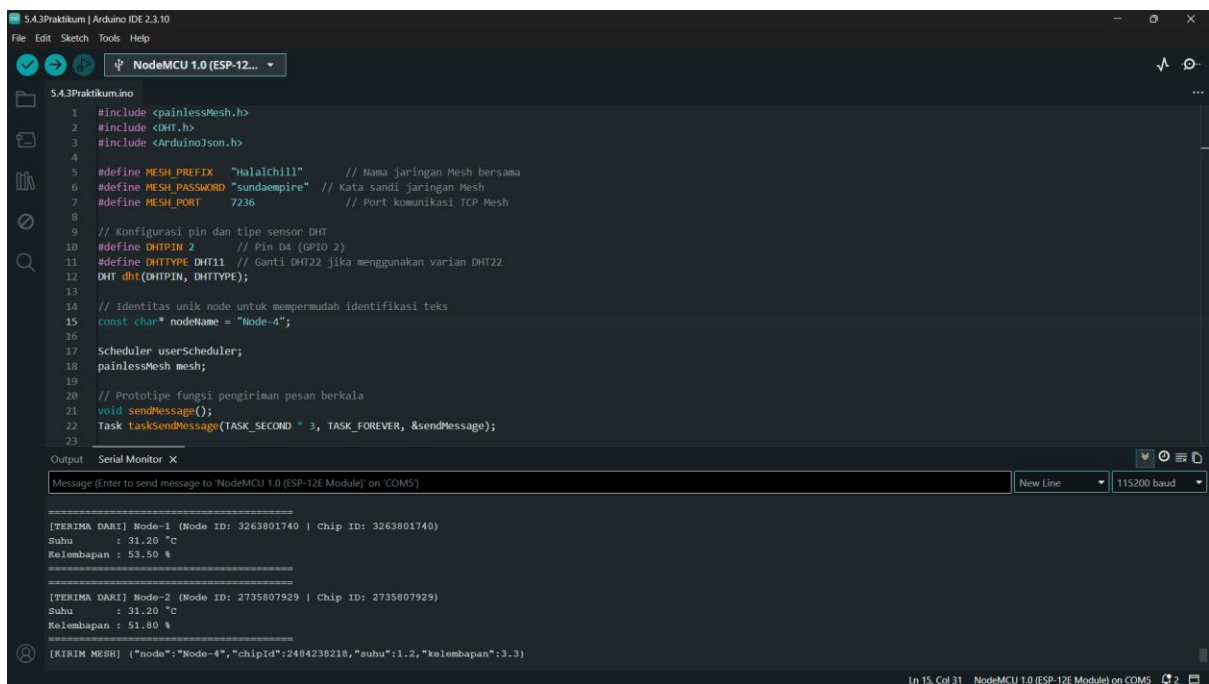
[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)  
Suhu : 31.20 °C  
Kelembapan : 53.40 %

=====

Building sketch

Ln 15, Col 31 NodeMCU 1.0 (ESP-12E Module) on COM7 2

## Penerima 3



```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Halalchill" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "sundaempire" // Kata sandi jaringan Mesh
7 #define MESH_PORT 7236 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-4";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
23
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

=====

[TERIMA DARI] Node-1 (Node ID: 3263801740 | Chip ID: 3263801740)  
Suhu : 31.20 °C  
Kelembapan : 53.50 %

=====

[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)  
Suhu : 31.20 °C  
Kelembapan : 51.80 %

=====

[KIRIM MESH] ["node":"Node-4","chipid":2484238218,"suhu":1.2,"kelembapan":3.3)

Ln 15, Col 31 NodeMCU 1.0 (ESP-12E Module) on COM5 2

**Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`)

ESP-NOW mendeteksi kegagalan pengiriman tanpa TCP melalui mekanisme MAC ACK frame di Layer 2 Wi-Fi (IEEE 802.11). Saat data dikirim secara *unicast*, chip radio penerima otomatis membalas dengan sinyal ACK di tingkat *hardware*. Jika penerima mati, sinyal ACK tidak terkirim, sehingga chip pengirim akan melakukan *retry* beberapa kali. Ketika batas percobaan habis tanpa balasan, pengirim menyimpulkan perangkat target tidak aktif dan melaporkan status gagal ke fungsi *callback* `OnDataSent`.

```
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM12')
TERIMA DARI] Node-3 (Node ID: 758106981 | Chip ID: 758106981)
Suhu      : 29.90 °C
Kelembapan : 51.00 %
=====
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":54.8}
=====
TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu      : 1.90 °C
Kelembapan : 2.20 %
=====
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":56.1}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":56.1}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":56.3}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":56}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":55}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":54.9}
--> Topologi rantai mesh telah diperbarui
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":53.7}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":53.3}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.2,"kelembapan":52.4}
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":52.4}
--> Topologi rantai mesh telah diperbarui
--> Koneksi Baru Terdeteksi! Node ID: 2735807929
KIRIM MESH] {"node":"Node-1","chipId":3263801740,"suhu":31.3,"kelembapan":52.6}
=====
TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)
Suhu      : 31.30 °C
Kelembapan : 51.30 %
=====
```

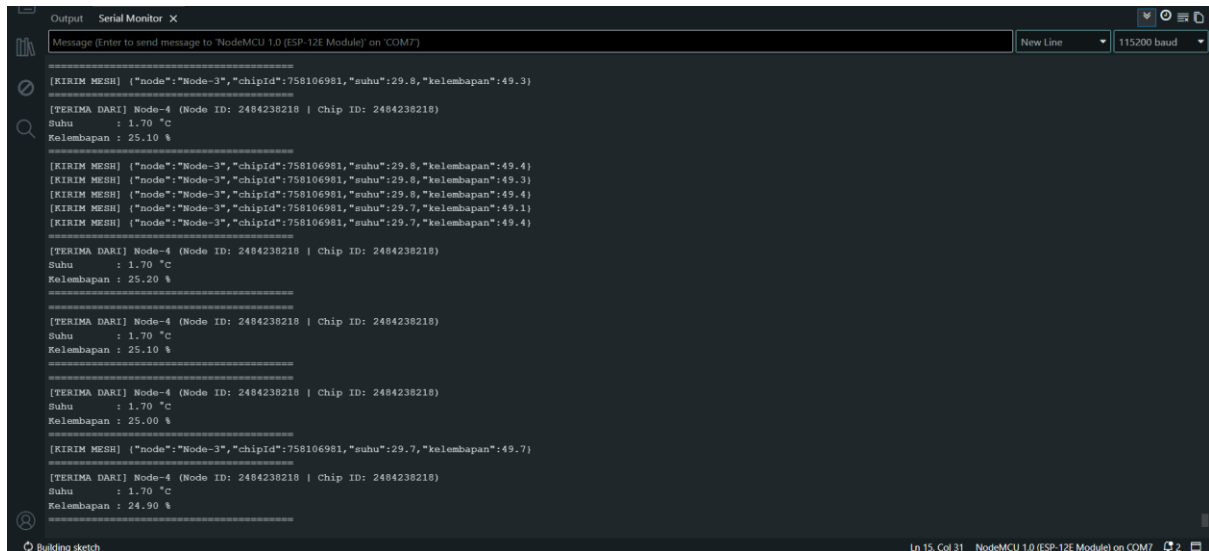
**Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan `painlessMesh` pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan `--> Koneksi Baru Terdeteksi! Node ID: ....`

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

5 Detik

- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

3 Detik



```
Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')
New Line 115200 baud

[KIRIM MESH] ("node":"Node-3","chipId":758106981,"suhu":29.8,"kelembapan":49.3)
[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.10 %

[KIRIM MESH] ("node":"Node-3","chipId":758106981,"suhu":29.8,"kelembapan":49.4)
[KIRIM MESH] ("node":"Node-3","chipId":758106981,"suhu":29.8,"kelembapan":49.3)
[KIRIM MESH] ("node":"Node-3","chipId":758106981,"suhu":29.8,"kelembapan":49.4)
[KIRIM MESH] ("node":"Node-3","chipId":758106981,"suhu":29.7,"kelembapan":49.1)
[KIRIM MESH] ("node":"Node-3","chipId":758106981,"suhu":29.7,"kelembapan":49.4)

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.20 %

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.10 %

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 25.00 %

[KIRIM MESH] ("node":"Node-3","chipId":758106981,"suhu":29.7,"kelembapan":49.7)

[TERIMA DARI] Node-4 (Node ID: 2484238218 | Chip ID: 2484238218)
Suhu : 1.70 °C
Kelembapan : 24.90 %

Building sketch Ln 15, Col 31 NodeMCU 1.0 (ESP-12E Module) on COM7
```

### Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON -

**Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

- Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- Jika suhu bernilai normal ( $\leq 32.0\text{ }^{\circ}\text{C}$ ), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetri dari Node <chipId> aman`.
- Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

5.4.3Praktikum | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...

5.4.3Praktikum.ino

```
103 // Atur level log debugging (hanya tampilkan error dan status startup)
104 mesh.setDebugTypes(ERROR | STARTUP);
105
106 // Inisialisasi jaringan mesh
107 mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
108
109 // Daftarkan event callback
110 mesh.onReceive(&receivedCallback);
111 mesh.onNewConnection(&newConnectionCallback);
112 mesh.onChangedConnections(&changedConnectionCallback);
113
114 // Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
115 userScheduler.addTask(taskSendMessage);
116 taskSendMessage.enable();
117
118 Serial.print("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
119 }
120
121 void loop() {
122 // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
123 mesh.update();
124 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

New Line 115200 baud

[TERIMA DARI] Node-2 (Node ID: 2735807929 | Chip ID: 2735807929)  
Suhu : 30.80 °C  
Kelembapan : 51.80 %  
\*\*\*\*\*  
[NORMAL] Telemetri dari Node 758106981 aman  
[KIRIM MESH] ("node":"Node-3","chipId":"2484238218","suhu":29.3,"kelembapan":51.4)  
[NORMAL] Telemetri dari Node 758106981 aman  
[KIRIM MESH] ("node":"Node-3","chipId":"2484238218","suhu":29.4,"kelembapan":52)  
[NORMAL] Telemetri dari Node 758106981 aman  
[KIRIM MESH] ("node":"Node-3","chipId":"2484238218","suhu":29.4,"kelembapan":52)  
[NORMAL] Telemetri dari Node 758106981 aman  
[KIRIM MESH] ("node":"Node-3","chipId":"2484238218","suhu":29.4,"kelembapan":51.8)

Ln 124, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM5